

LUMEN · Especificación Técnica de Implementación

Cotejo de baseline, delta Supabase, prompt Coder React, validaciones y smokes

Versión final ajustada V5 · 21 de julio de 2026

1. Alcance y fuentes verificadas

Esta versión sustituye la especificación técnica anterior. Fue cotejada contra DB - MVP BASE (19-7-2026), LUMEN_archivos_base_integrados (21-7-26) y LUMEN_componentes_integrados (21-7-26). El coder modifica exclusivamente React. Los scripts Supabase y las consultas de validación son definidos por ChatGPT y ejecutados con Pablo.

2. Veredicto de implementabilidad

VEREDICTO: IMPLEMENTABLE. No requiere reconstruir el motor ni crear un sistema paralelo. El check-in y el aprendizaje por capacidades ya están mayormente resueltos. El delta estructural principal es evolucionar user_area_faros de “un texto por área” a “varios Faros personales con ciclo de vida”.

3. Baseline real confirmado

Pieza vigente	Estado confirmado
React useLumi	Ya conserva checkinHemisphere, checkinState, checkinArea, checkinFaro, checkinCapability y checkinTime y los envía al dispatcher.
checkin_options	Renderer genérico ya despacha submit_checkin_<step>; effect usa perceived_capability_key.
experience_hypotheses	Ya relaciona experiencia, hemisferio, área, estado, Faro, capacidad seleccionable y capacidad esperada.
experience_runs	Ya persiste selected_capability_key, expected_capability_key, perceived_capability_key, hypothesis_json, experience_hypothesis_id, área, Faro y user_area_faro_id.
sanctuary_items	Ya conserva área, Faro, capacidades, hipótesis, reflexión, señal y vínculo al run.
Feedback	lumi_submit_feedback_effect y lumi_complete_experience_run ya aceptan perceived_capability_key.
Catálogos	ui_copy ya contiene dominios area, faro, capability, state y time_bucket.
Faros actuales	user_area_faros contiene id, user_id, area, faro_text, faro_key y updated_at.
Restricción actual	uq_user_area_faros UNIQUE(user_id, area): hoy impide varios Faros por área.
UI Faros actual	FarosPanel usa textarea y save_faro onBlur; representa el modelo anterior de texto libre.
Entrada	App ya usa LumiOrb en vistas simples y normales; no existe LumenIntro en los archivos integrados.

4. Inconsistencias detectadas y resolución final

Hallazgo	Riesgo	Resolución
La especificación anterior citaba baseline 16-7	Podía proponer cambios ya realizados.	Baseline actualizado a DB 19-7 y React 21-7.
H1 actual pasa por scan → área → estado	Contradice el criterio de baja carga y la especificación funcional.	Preservar LandingScan, pero su continuación debe ir a estado; área deja de ser obligatoria en H1.
H2 muestra todas las etiquetas del catálogo	Confunde etiqueta disponible con Faro activo personal.	Separar activación de selección: H2 muestra user_area_faros activos; activación muestra ui_copy domain=faro.
Seleccionar Faro en H2 solo guarda faro_key en sesión	Puede iniciar runs sin user_area_faro_id y debilitar el recorrido.	La selección debe recibir/validar user_area_faro_id y persistir ambos.
UNIQUE(user_id, area)	Impide varios Faros por área.	Reemplazar por unicidad compatible con varios Faros: user_id + area + faro_key para Faros catalogados.
user_area_faros no tiene status ni cierre	No permite pausar/cerrar.	Agregar ciclo de vida mínimo y timestamps.
FarosPanel usa texto libre	No implementa etiquetas ni múltiples Faros.	Reutilizar content_type faros_panel con contrato declarativo nuevo; no crear otro content type.
Capacidades del check-in se muestran completas	Puede mostrar opciones sin cobertura contextual.	Filtrar desde experience_hypotheses + experiencia/recurso ejecutable.
Tiempo siempre muestra tres buckets	Puede producir camino muerto.	Mostrar solo buckets que conservan una UA ejecutable.
Faros legacy con faro_key null	Una migración agresiva los perdería o inventaría equivalencias.	Conservarlos; no mapear automáticamente. Quedan visibles como historia y fuera de nuevas activaciones por etiqueta.
La animación presupone activo gráfico	Un morph exacto no es posible sin trazado oficial.	Requerir SVG/trazado oficial; fallback técnico: reveal/crossfade hacia LumiOrb, sin tipografía aproximada.

5. Módulo A · Check-in e hipótesis disponibles

5.1 Funciones a preservar y ajustar

Función	Delta
lumi_submit_checkin_hemisphere	H1 conserva LandingScan; H2 conserva entrada contextual/reanudación.
Continuación de LandingScan / nodo H1	Cambiar destino funcional de área a estado, sin tocar el componente visual.
lumi_submit_checkin_state	Devolver solo capacidades H1 con cobertura ejecutable para el estado elegido.
lumi_submit_checkin_area	En H2 devolver Faros activos personales; si no hay, devolver flujo declarativo de activación.
lumi_submit_checkin_faro	Validar user_area_faro_id, usuario, área y status=active; persistir user_area_faro_id + faro_key.
lumi_submit_checkin_capability	Validar cobertura y devolver solo time_bucket viables.
lumi_submit_checkin_time	Iniciar experiencia solo si la combinación continúa cubierta.
lumi_start_experience / lumi_select_experience	Mantener persistencia ya existente de hipótesis y tres contextos de capacidad.
lumi_submit_feedback_effect	Sin rediseño; preservar perceived_capability_key y divergencias.

5.2 Regla de disponibilidad

Una opción se considera disponible únicamente cuando existe experience_hypotheses.active=true compatible con el contexto ya elegido, vinculada a una experiencia activa/canónica y a un recurso activo, utilizable y compatible con el

tiempo. Los campos nulos de la hipótesis funcionan como alcance general; los valores específicos aumentan la pertinencia, no deben bloquear hipótesis generales.

5.3 Ajuste importante de cobertura

El catálogo actual tiene hipótesis generales y específicas. Por ello el filtrado debe aceptar coincidencia exacta o NULL para área, estado y Faro, priorizando la coincidencia específica. Exigir siempre state_key/faro_key exactos eliminaría cobertura válida y degradaría el motor.

5.3.1 Cobertura real confirmada en DB

La tabla experience_hypotheses contiene 201 hipótesis activas: 91 H1 y 110 H2. En todas ellas hemisphere_key, capability_key y expected_capability_key están informados.

H1: 20 hipótesis poseen state_key específico, cubriendo 7 estados; 16 poseen life_area_key específico, cubriendo 5 áreas; las restantes funcionan como hipótesis generales mediante NULL. Ninguna hipótesis H1 utiliza Faro, como corresponde.

H2: 20 hipótesis poseen faro_key específico, cubriendo 13 etiquetas; 25 poseen life_area_key específico, cubriendo 6 áreas; las restantes funcionan como hipótesis generales mediante NULL. H2 no utiliza state_key.

Tiempo: experience_hypotheses no contiene time_bucket. La disponibilidad temporal se obtiene de la experiencia y de sus recursos activos. Las 201 hipótesis verificadas conducen a recursos con duración disponible entre 2 y 15 minutos. Por lo tanto, el filtro de tiempo es implementable, pero debe resolverse mediante JOIN con experiencias/recursos y traducción a los buckets vigentes.

Conclusión técnica: existen todos los datos necesarios para el filtrado progresivo, pero el algoritmo correcto debe combinar atributos específicos, NULL como wildcard compatible y duración real del recurso. No corresponde exigir una fila de hipótesis por cada combinación cartesiana de hemisferio, área, estado, Faro, capacidad y tiempo.

5.4 Prompt Coder React · Check-in

Modificar solo React para consumir los contratos declarativos actualizados. Mantener useLumi y buildParams como fuente única del estado de check-in. No calcular cobertura, no consultar tablas, no seleccionar hipótesis y no hardcodear transiciones. Preservar LandingScan y hacer que despache la action recibida. Mantener checkin_options genérico y perceived_capability_key para step=effect. No tocar Supabase. Ejecutar build/typecheck y reportar archivos modificados y grep de actions afectadas.

5.5 Validaciones

- Cobertura por cada estado H1 y capacidad devuelta.
- Cobertura por cada Faro activo H2 y capacidad devuelta.
- Time buckets devueltos con al menos un recurso compatible.
- Cero runs nuevos H2 con user_area_faro_id nulo.
- Persistencia de selected/expected/perceived capability.
- Divergencia válida entre capacidades.
- Fallback sobrio cuando no existe match.

6. Módulo B · Faros y Santuario Full

6.1 Migración mínima de user_area_faros

6.1.1 Limpieza aprobada de datos de prueba

Antes de alterar restricciones o contratos se eliminan los Faros personales actuales, dado que son datos de prueba. Esta limpieza no elimina catálogo, hipótesis, experiencias, runs ni elementos del Santuario.

SQL de limpieza: DELETE FROM public.user_area_faros; La ejecución debe realizarse dentro de la migración transaccional y validarse con SELECT count(*) FROM public.user_area_faros; esperado: 0.

Cambio	Decisión
Eliminar uq_user_area_faros UNIQUE(user_id, area)	Necesario para permitir varios Faros por área.
status text default active	Valores permitidos: active, paused, closed.
activated_at timestamptz	Para existentes: COALESCE(updated_at, now()).
paused_at timestamptz	Última pausa.
closed_at timestamptz	Cierre del tramo.
closing_sentiment text	positive, neutral, negative.
closing_value text	Capitalización elegida.
closing_reflection text	Reflexión opcional.
faro_key nullable	Solo para preservar Faros legacy; obligatorio en nuevas activaciones.
Índice único parcial	UNIQUE(user_id, area, faro_key) WHERE faro_key IS NOT NULL AND status <> closed, o regla equivalente validada antes del script final.
Índices de lectura	user_id + status; user_id + area + status; user_id + faro_key.

Tras limpiar los datos de prueba, la restricción y el modelo de unicidad deben permitir varios Faros por área, impedir duplicados activos de la misma etiqueta para una persona y conservar el ciclo active / paused / closed. No se requiere backfill ni mapeo de texto libre.

6.2 RPCs

RPC	Responsabilidad
lumi_get_faros	Evolucionar contrato: agrupación por área/estado, Faros legacy y catálogo activable.
lumi_activate_faro	Crear o reactivar Faro catalogado; nunca aceptar texto libre nuevo.
lumi_pause_faro	active → paused.
lumi_resume_faro	paused → active.
lumi_begin_close_faro	Nodo/formulario declarativo para integración final.
lumi_close_faro	Persistir cierre y devolver síntesis factual.
lumi_get_faro_journey	Reconstruir recorrido por user_area_faro_id desde runs y elementos asociados.
lumi_get_sanctuary / open_sanctuary	Preservar contrato de item_list vigente e incorporar acceso a Territorio/Faros y Recorridos sin romper consumidores.
lumi_save_faro	Mantener solo como compatibilidad legacy; no usar para nuevas activaciones por etiqueta.

6.3 Contratos de UI reutilizables

- faros_panel: recibe áreas, Faros personales por estado y etiquetas activables; despacha actions declaradas.
- item_list: continúa listando guardados y se reutiliza para recorridos.
- activity_detail: se reutiliza para detalle de Faro/recorrido y cierre cuando corresponda.
- note_editor: continúa para notas y reflexión donde resulte suficiente.
- No crear sanctuary_dashboard, faro_manager ni content types equivalentes.

6.3.1 Entrada declarativa del Santuario

Supabase debe resolver una entrada contextual del Santuario. El payload puede reutilizar item_list con source = sanctuary_home y hasta dos elementos reales relevantes. React no construye la introducción, no elige prioridades y no inventa resúmenes.

Las actions visibles se construyen dinámicamente desde cuatro señales: has_territory_or_faros, has_journeys, has_saved_items y has_reflections. Solo se devuelve la action de una dimensión cuando su señal es verdadera. Los nombres funcionales pueden ser open_sanctuary_territory, open_sanctuary_journeys, open_sanctuary_saved y

open_sanctuary_notes, o los nombres reales consolidados en Supabase. React consume exactamente las actions recibidas.

El orden de las actions lo decide Supabase. React no reordena, no completa y no suprime.

La respuesta puede contener de cero a cuatro actions. Cero actions de sección activa el estado inicial del Santuario; cuatro actions se admiten solo cuando las cuatro señales son verdaderas.

El mensaje de LUMI, los items contextuales y las actions deben derivarse de las mismas señales y consultas para evitar contradicciones.

has_reflections = existe al menos una nota o reflexión real en los orígenes incluidos por el modelo MVP.

has_saved_items = existe al menos un sanctuary_item vigente para la persona.

has_journeys = existe al menos un experience_run H2 vinculado inequívocamente a un user_area_faro_id.

has_territory_or_faros = existe al menos un user_area_faro activo, pausado o cerrado. Si es falso, puede ofrecerse una action específica de onboarding de Faros, pero no una pill de sección vacía.

Supabase calcula las cuatro señales en una única resolución del nodo de entrada.

6.3.1.a Resolución contextual y reglas de visibilidad

6.3.2 Contrato Territorio y Faros

faros_panel debe recibir áreas y Faros ya resueltos por Supabase, agrupados por status. Cada Faro debe incluir como mínimo user_area_faro_id, area_key, area_label, faro_key, faro_label, status y actions declaradas. Las etiquetas activables incluyen faro_key, label y action. React solo renderiza y despacha.

6.3.3 Contrato Recorridos

La lista de recorridos reutiliza item_list. Cada item representa un user_area_faro_id, no un resource_id. El detalle reutiliza activity_detail y recibe timeline prearmado por Supabase. React no agrupa runs, no calcula conteos, no deduce capacidades y no redacta síntesis.

6.3.4 Contrato Guardado para mí

Reutiliza el flujo actual de sanctuary_items y SanctuaryDetail. El cambio es únicamente de acceso/organización dentro del Santuario. No modificar contratos vigentes de abrir, editar nota, quitar, compartir o replay salvo adaptación mínima al nuevo nodo de entrada.

6.3.5 Contrato Notas y reflexiones

Reutiliza item_list para la agregación y note_editor para edición. Supabase devuelve cada nota con id contextual, origin_type, origin_id, title, excerpt, date, action y edit_action cuando corresponda. React no crea una store paralela ni fusiona textos por su cuenta.

6.3.6 Estados vacíos y fallbacks

Cada sección debe contar con nodo vacío propio para accesos internos o cambios concurrentes, pero la entrada principal no debe mostrar pills hacia dimensiones vacías. Los componentes no generan copy ni acciones alternativas. Un error o ausencia de datos no debe enviar a Home silenciosamente ni mostrar una lista vacía sin explicación.

6.4 Prompt Coder React · Santuario/Faros

Implementar exclusivamente en React la nueva organización contextual del Santuario definida en las especificaciones adjuntas. 1) Renderizar la entrada viva usando el mensaje, item_list y actions recibidos de Supabase; no crear dashboard ni navegación paralela. 2) No hardcodear cuatro accesos ni ocultarlos en React: mostrar exactamente las pills devueltas por Supabase, que ya excluyen dimensiones vacías. 3) Evolucionar FarosPanel: eliminar

textarea/onBlur para nuevas activaciones; renderizar áreas, Faros activos/pausados/cerrados y etiquetas activables según el payload de Supabase; despachar exactamente las actions recibidas con user_area_faro_id, faro_key y area_key. 4) Recorridos: reutilizar item_list para la lista y activity_detail para la cronología; no agrupar runs ni calcular síntesis en React. 5) Guardado para mí: preservar íntegramente SanctuaryDetail, apertura, edición de nota, quitar, compartir y replay. 6) Notas y reflexiones: reutilizar item_list y note_editor; editar el registro original recibido; no crear estado persistente paralelo ni notas sueltas. 7) Implementar estados vacíos exclusivamente con nodos/payloads recibidos. No crear SQL, RPCs, content types específicos ni lógica de negocio. No modificar Fuente, feedback, compartir, cuenta ni check-in. Ejecutar build, typecheck y smokes de regresión.

6.5 Regresión obligatoria del Santuario

- La entrada contextual muestra un mensaje de LUMI coherente con los datos reales y hasta dos elementos significativos.
- Solo aparecen pills para dimensiones con contenido; React no aplica filtros locales ni agrega accesos faltantes.
- Cuando las cuatro dimensiones poseen contenido pueden mostrarse cuatro pills conversacionales como excepción explícita a la regla habitual de tres.
- Territorio y Faros activa, pausa, reactiva y cierra sin depender de texto libre ni decisiones React.
- Recorridos lista Faros y abre cronología real; no aparecen sesiones H1 ni datos inventados.
- Quitar un elemento de Guardado para mí no elimina su experience_run ni su recorrido.
- La lista de guardados del Santuario permanece íntegra; únicamente se eliminan registros de prueba de user_area_faros.
- Detalle de guardado conserva badges de capacidad y Faro.
- Agregar/editar nota funciona.
- Quitar funciona.
- Compartir luz funciona.
- Los Faros personales de prueba se eliminan antes de la migración; no se implementa compatibilidad legacy.
- Faros pausados/cerrados no desaparecen del Santuario.

7. Módulo C · Entrada animada

7.1 Implementación

- Nuevo componente local LumenIntro.
- Integración en App.tsx por encima de la vista, sin frenar bootstrap/useLumi.
- sessionStorage para reproducir una vez por sesión de navegación.
- Reutilizar LumiOrb como estado final.
- SVG/trazado oficial obligatorio para la palabra Lumen.
- prefers-reduced-motion y skip.
- Sin cambios en useLumi, Dispatcher, nodos o Supabase.

7.2 Prompt Coder React · Entrada

Crear LumenIntro con estados visuales reveal → dissolve → orb → complete. Usar el SVG/trazado oficial existente o incorporado al repositorio; no reproducir el isologo con una fuente genérica. Montarlo en App.tsx una vez por sessionStorage, mientras useLumi carga normalmente. Permitir skip, respetar prefers-reduced-motion y fallback directo a LumiOrb. Evitar dependencias pesadas. No tocar Supabase ni flujos. Validar desktop, móvil y build/typecheck.

8. Orden de implementación y gates

Fase	Gate
0 · Baseline	Guardar resultados actuales de H1, H2, Santuario, compartir y

	cuenta.
1 · Check-in Supabase	Sin caminos muertos; H1 sin área obligatoria; runs completos.
2 · Check-in React	LandingScan y checkin_options consumen contratos sin lógica nueva.
3 · Faros Supabase	Múltiples Faros, estados y recorrido íntegro; legado preservado.
4 · Santuario React	Gestión por etiquetas y estados sin degradar guardados/notas/compartir.
5 · Entrada React	Animación no bloqueante y accesible.
6 · Regresión	Matriz completa aprobada antes de validación con terceros.

9. Matriz de smokes

Smoke	Resultado esperado
H1	Entrada → scan → estado → capacidad → tiempo → UA → feedback; no pide área.
H1 sin cobertura específica	Se utilizan hipótesis generales compatibles o fallback, nunca error.
H2 con Faro activo	Área → Faro personal → capacidad → tiempo → UA; run con user_area_faro_id.
H2 sin Faro	Ofrece activar etiqueta; luego continúa al Faro recién activado.
Múltiples Faros	Dos Faros en la misma área aparecen y generan recorridos separados.
Pausa/reactivación	Sale/vuelve a H2 sin perder recorrido.
Cierre	Acepta valoración positiva, neutra o negativa y conserva síntesis.
Legacy	Faro de texto libre previo sigue visible y no se pierde.
Feedback capacidades	selected, expected y perceived pueden diferir.
Santuario regresión	Abrir, nota, quitar, replay y compartir funcionan.
Fuente/cuenta	Navegación y cuenta express no se alteran.
Entrada	Una vez por sesión; skip; reduced-motion; sin bloquear carga.

10. Riesgo residual y control

No se detecta un bloqueo estructural. Los riesgos principales son de migración y contrato: la restricción única de Faros, los registros legacy sin faro_key, el cambio de H1 para omitir área y la necesidad de conservar los contratos actuales del Santuario. Todos son controlables mediante prechecks, migración transaccional, postchecks y smokes antes de habilitar React.

11. Definición técnica de cierre

La implementación queda cerrada cuando Supabase sigue siendo la única capa que decide opciones e hipótesis, React se limita a render y despacho, los Faros poseen identidad personal y ciclo de vida, los recorridos se reconstruyen desde relaciones existentes, los datos legacy permanecen intactos y la regresión integral demuestra que no se rompió ni degradó ninguna capacidad vigente.